

Projet d'algorithmique CIN1

Jeu d'échec

Table des matières

I - Présentation du sujet	2
II - Rendu visuel	2
III - Constantes et fonctions fournies	4
a) Constantes	4
b) Fonctions fournies	5
IV - Fonctions à implémenter	5
a) Fonction main	6
b) Initialisation et affichage	6
c) Gestion des couleurs, saisie d'un coup	7
d) Implémentation du jeu d'échec	8
V - Listes des fonctionnalités en bonus	10

I - Présentation du sujet

Le but de ce projet est de réaliser en C un jeu d'échec interactif, jouable dans un terminal d'un système Linux.

Il faudra mettre en pratique les notions vues en cours, en particulier les fonctions, les tableaux unidimensionnels et les tableaux à 2 dimensions.

Note sur l'évaluation du projet : une soutenance aura lieu fin janvier lors de laquelle il faudra présenter et expliquer votre travail. De plus, il est impératif que vous rendiez votre code pour la date qui vous sera communiquée, que ce code respecte les consignes listées ci-dessous (en particulier respecte les prototypes des fonctions) et que le code compile. Un groupe dont le code rendu ne compile pas ne peut avoir la moyenne – si besoin commentez les bouts de codes que vous n'arrivez pas à compiler.

Quelques références utiles :

- Règles du jeu :
https://fr.wikipedia.org/wiki/%C3%89checs#R%C3%A8gles_du_jeu
- Bonus : échecs Fischer 960 :
https://fr.wikipedia.org/wiki/%C3%89checs_al%C3%A9atoires_Fischer

II - Rendu visuel

L'échiquier sera modélisé par un tableau 2D de caractères de 8x8 cases, et les pièces des deux joueurs seront encodés par des caractères spécifiques. Dans l'ordre, les tour, fou, dame, roi, cavalier et pion seront représentés par :

Joueur blanc : T, F, D, R, C, P

Joueur noir : t, f, d, r, c, p

La casse des caractères est différente suivant la couleur du joueur afin de pouvoir aisément reconnaître les pièces de chacun. En plus de cela, il sera conseillé de réaliser un affichage en couleur à l'aide d'une fonction qui vous est fournie. Un exemple de rendu possible du début d'une partie du jeu d'échec :

```

File Edit View Search Terminal Help

Coup: 0
Tour du joueur: blanc
Temps: 0 s

+---+---+---+---+---+---+---+
8 | t | c | f | d | r | f | c | t |
+---+---+---+---+---+---+---+
7 | p | p | p | p | p | p | p | p |
+---+---+---+---+---+---+---+
6 |   |   |   |   |   |   |   |   |
+---+---+---+---+---+---+---+
5 |   |   |   |   |   |   |   |   |
+---+---+---+---+---+---+---+
4 |   |   |   |   |   |   |   |   |
+---+---+---+---+---+---+---+
3 |   |   |   |   |   |   |   |   |
+---+---+---+---+---+---+---+
2 | P | P | P | P | P | P | P | P |
+---+---+---+---+---+---+---+
1 | T | C | F | D | R | F | C | T |
+---+---+---+---+---+---+---+
  A B C D E F G H

Pièces blanches capturées:
Pièces noires capturées:

Saisir un coup! Coordonnées de depart, et d'arrivee: |

```

Un joueur choisit son coup en rentrant les coordonnées de la case de départ de son coup, suivies par celles de la case d'arrivées. Les coordonnées d'une case seront données par son nom de colonne (A-H) suivi par son nom de ligne (1-8), voir l'exemple ci-dessous.

En bonus, vous pourrez faire en sorte que l'échiquier soit tourné lorsque c'est au joueur noir de jouer, afin de voir la partie de son point de vue :

```

File Edit View Search Terminal Help

Coup: 1
Tour du joueur: noir
Temps: 2 s

+---+---+---+---+---+---+---+
1 | T | C | F | R | D | F | C | T |
+---+---+---+---+---+---+---+
2 | P | P | P |   | P | P | P | P |
+---+---+---+---+---+---+---+
3 |   |   |   |   |   |   |   |   |
+---+---+---+---+---+---+---+
4 |   |   |   | P |   |   |   |   |
+---+---+---+---+---+---+---+
5 |   |   |   |   |   |   |   |   |
+---+---+---+---+---+---+---+
6 |   |   |   |   |   |   |   |   |
+---+---+---+---+---+---+---+
7 | p | p | p | p | p | p | p | p |
+---+---+---+---+---+---+---+
8 | t | c | f | r | d | f | c | t |
+---+---+---+---+---+---+---+
  H G F E D C B A

Pièces blanches capturées:
Pièces noires capturées:

Saisir un coup! Coordonnées de depart, et d'arrivee: E7E6|

```

En bonus aussi, vous pourrez afficher les listes des pièces capturées par les deux joueurs comme ci-dessous, et le roi en rouge s'il est mis en échec (ici par la dame noire) :

```

File Edit View Search Terminal Help
Coup: 10
Tour du joueur: blanc
Temps: 346 s

+---+---+---+---+---+---+---+
8 | t | c | f |   | r | f | c | t |
+---+---+---+---+---+---+---+
7 | p | p |   | p |   | p | p | p |
+---+---+---+---+---+---+---+
6 |   |   |   |   | p |   |   |   |
+---+---+---+---+---+---+---+
5 |   | p |   |   |   |   |   |   |
+---+---+---+---+---+---+---+
4 |   |   |   | P | P |   |   |   |
+---+---+---+---+---+---+---+
3 |   |   | C |   |   | C |   |   |
+---+---+---+---+---+---+---+
2 | P | P | P |   | d | P | P |   |
+---+---+---+---+---+---+---+
1 | T |   | F | D | R |   |   | T |
+---+---+---+---+---+---+---+
  A  B  C  D  E  F  G  H

Pièces blanches capturees: F P
Pièces noires capturees:

Vous etes en echec!

Saisir un coup! Coordonnees de depart, et d'arrivee: |

```

On remarquera qu'à chaque nouveau coup, l'affichage de la console est effacé afin de ne garder à l'écran que l'essentiel (voir plus bas comment faire cela).

III - Constantes et fonctions fournies

Ci-dessous sont listées les constantes et les fonctions déjà implémentées qui vous sont fournies. Il faudra utiliser tout cela lors de l'implémentation des fonctions restantes.

a) Constantes

Option pour tourner l'échiquier lorsque le joueur noir est en train de jouer :

```
#define TOURNER_ECHIQUEL false
```

Contenu d'une case vide de l'échiquier :

```
#define CASE_VIDE ' '
```

L'échiquier sera modélisé par un tableau 2D de caractères, de taille NxN.

```
#define N 8
```

Nombre maximal de pièces d'une couleur pouvant être capturées.

```
#define NB_MAX_CAPTURES 15
```

Nombre maximal de coup possible pour une pièce (27 pour la dame pour une grille 8x8). Ne pas modifier :

```
#define NB_MAX_DEPL (4*(N)-5)
```

Encodage des couleurs d'une case de l'échiquier :

```
#define C_BLANC 0
```

```
#define C_NOIR 1
```

```
#define C_VIDE 2
```

Encodage de l'état en cours d'une partie :

```
#define NON_TERMINEE 0
```

```
#define DEFAITE_BLANC 1
```

```
#define DEFAITE_NOIR 2
```

```
#define PARTIE_NULLE 3
```

b) Fonctions fournies

```
void effaceConsole(void);
```

Fonction pouvant être utilisée pour afficher les cases de l'échiquier. Si personne n'est en échec, passer pour couleur C_VIDE.

```
void afficheCaseEnCouleur(char c, int couleurEnEchec);
```

Fonctions à implémenter

Votre mission est d'implémenter les fonctions suivantes, en respectant leur prototype et les consignes données pour chacune. Si vous souhaitez rajouter des fonctionnalités en bonus, vous pourrez si besoin vous servir de variables globales.

a) Fonction main

Dans la fonction *main()* à implémenter, on commencera par initialiser l'échiquier avec *initialiseEchiquier()*, puis on lancera la boucle principale du programme. On ne sortira de cette boucle que lorsque la partie sera terminée. À l'intérieur de la boucle, il faudra :

- Afficher l'échiquier à l'aide de *afficheEchiquier()*.
- Réaliser la saisie d'un coup valide et mettre à jour l'échiquier via *saisieCoup()*.
- Evaluer à l'aide de *partieTerminee()* si la partie est terminée ou continue.

À l'issue de la boucle, on affichera une dernière fois l'échiquier et on affichera aussi un message commentant le résultat de la partie.

b) Initialisation et affichage

Renvoie *true* si la case de coordonnées (ligne, colonne) est présente dans la grille, et *false* sinon.

```
bool estDansGrille(int ligne, int colonne);
```

Renvoie *true* si la case (ligne, colonne) est dans la grille et est vide.

```
bool estCaseVide(const char grille[N][N], int ligne, int colonne);
```

Remplie l'échiquier par le caractère CASE_VIDE.

```
void videGrille(char grille[N][N]);
```

Recopie l'intégralité d'une grille dans une autre.

```
void copieGrille(char copie[N][N], const char grille[N][N]);
```

Initialise la grille à partir de la position standard du jeu d'échec. On pourra utiliser la fonction *videGrille()* au préalable.

En bonus : initialisation aléatoire de l'échiquier façon Fischer 960.

```
void initialiseEchiquier(char grille[N][N]);
```

On pourra effacer la console avec *effaceConsole()* et utiliser la fonction

afficheCaseEnCouleur() pour obtenir un affichage plus plaisant. En bonus:

- Afficher le roi blanc ou noir en rouge s'il est mis en échec.
- Si `TOURNER_ECHIQUE` est true, l'échiquier doit être tourné lorsque le joueur noir joue, c'est à dire que l'affichage des lignes et des colonnes de la grille se fait dans le sens inverse. On fera aussi attention à l'affichage des légendes des lignes et colonnes.
- Afficher les listes des pièces capturées par les deux joueurs.

```
void afficheEchiquier(const char grille[N][N], int num_coup, size_t  
temps_debut);
```

c) Gestion des couleurs, saisie d'un coup

Renvoie la couleur adverse de la couleur donnée (on supposera la couleur différente de `C_VIDE`).

```
int couleurAdverse(int couleur);
```

Renvoie `C_VIDE` si la case est hors de la grille ou est vide, `C_BLANC` si la case contient une pièce blanche, et `C_NOIR` si c'est une pièce noire.

```
int trouveCouleur(const char grille[N][N], int ligne, int colonne);
```

Renvoie true si le caractère est une majuscule, et false sinon.

```
bool estMajuscule(char c);
```

Convertit la notation d'une case en coordonnées de la forme (ligne, colonne).

Un exemple : A8 -> (0, 0)

Le résultat est écrit dans le tableau de coordonnées. On pourra utiliser les fonctions *estMajuscule()* afin que la saisie d'une colonne ne dépende pas du choix de sa casse, et on s'assurera avec *estDansGrille()* que la case convertie est bien présente dans la grille. Si c'est le cas, la fonction renverra *true*, et *false* sinon.

```
bool convertitEnCoordonnees(const char notation[2], int coordonnees[2]);
```

Fait réaliser à l'utilisateur la saisie au clavier d'un coup donné par sa case de départ et la case d'arrivée. Une case sera donnée par son nom de colonne (A-H) suivi par son nom de ligne (1-8). Un exemple de saisie possible en début de partie : E2E4. On devra réaliser une boucle de saisie, jusqu'à obtenir de l'utilisateur des coordonnées valides, et définissant un coup valide. Il faudra appeler les fonctions suivantes :

- *convertitEnCoordonnees()* pour obtenir les coordonnées de la case de départ et de la case d'arrivée.

- *realiseCoupSiValide()* pour effectuer le coup à condition qu'il soit valide.

```
void saisieCoup(char grille[N][N]);
```

d) Implémentation du jeu d'échec

Cherche dans l'échiquier la position du roi de la couleur donnée, et sauvegarde le résultat dans le tableau *positionRoi*.

```
void trouvePositionRoi(const char grille[N][N], int positionRoi[2], int couleur);
```

Déplace la pièce de la position de départ vers l'arrivée. La case de départ doit être vidée. En bonus : la promotion d'un pion, le roque, et la prise en passant. Attention, on ignorera le fait que le coup soit valide ou non.

```
void realiseCoup(char grille[N][N], const int depart[2], const int arrivee[2]);
```

On réalise le coup choisit s'il est valide, c'est dire si le déplacement de la pièce en case de départ vers l'arrivée est valide, et si à l'issue de ce déplacement le roi de la couleur jouée n'est pas en échec. Si le coup n'est pas valide, il ne faudra pas modifier l'échiquier. On devra appeler les fonctions *estDeplacementValide()*, *estCoupValide()*.

```
bool realiseCoupSiValide(char grille[N][N], const int depart[2], const int arrivee[2]);
```


Analyse l'échiquier afin de savoir si le roi de la couleur donnée est mis en échec. Pour ce faire, il faudra trouver la position du roi, et parcourir chaque case de la grille à la recherche des pièces de la couleur adverse. La fonction renvoie *true* si une de ces pièces a pour déplacement valide la case sur laquelle le roi concerné se trouve, et *false* sinon. On appellera les fonctions : *trouvePositionRoi()*, *trouveCouleur()*, *couleurAdverse()*, *estDeplacementValide()*.

```
bool estEnEchec(const char grille[N][N], int couleur);
```

On dit qu'un coup est valide si le déplacement de la pièce en case de départ vers la case d'arrivée est valide, et si ce déplacement ne met pas le roi de la couleur de la pièce de départ en échec. On supposera dans cette fonction que le déplacement considéré a déjà été validé. Si la case de départ est vide, la fonction renverra *false*. Sinon, on créera une copie de l'échiquier dans laquelle on réalisera le coup considéré. S'il y a alors échec le coup n'est pas valide, sinon il l'est. On appellera les fonctions : *estCaseVide()*, *copieGrille()*, *realiseCoup()*, *trouveCouleur()*, *estEnEchec()*.

```
bool estCoupValide(const char grille[N][N], const int depart[2], const int arrivee[2]);
```

Renvoie *true* si le déplacement est licite pour la pièce en case de départ, indépendamment du fait que le roi de la même couleur soit en échec à l'issue de ce déplacement. Pour ce faire, on listera avec *listeDeplacementsValides()* l'ensemble des déplacements valides et on vérifiera si la case d'arrivée figure parmi ces déplacements.

```
bool estDeplacementValide(const char grille[N][N], const int depart[2], const int arrivee[2]);
```

Détecte si une partie est terminée (victoire, défaite, ou partie nulle) ou non. On parcourra toutes les pièces du joueur donné, et pour chacune on listera l'ensemble de leur déplacements valides. On comptera alors parmi ces déplacements lesquels sont des coups valides. S'il n'y a plus aucun coup valide pour le joueur, soit il est en échec et a perdu, soit la partie est nulle. Suivant les

différents cas, la fonction doit renvoyer l'entier suivant :

- NON_TERMINEE : partie non terminée.
- DEFAITE_BLANC : joueur blanc a perdu.
- DEFAITE_NOIR : joueur noir a perdu.
- PARTIE_NULLE : pat, i.e partie nulle.

On appellera les fonctions *trouveCouleur()*, *listeDeplacementsValides()*, *estCoupValide()*, *estEnEchec()*.

En bonus : gérer la règle de la triple répétition et la règle des 50 coups sans capture.

```
int partieTerminee(const char grille[N][N], int couleur);
```

Trouve tous les déplacements valides de la pièce choisie, sans prendre en compte la mise en échec, et remplit le tableau '*liste*' avec ces coups. Le nombre de coups trouvés est renvoyé. On réalisera un filtrage par type de pièce jouée : tour, fou, dame, roi, cavalier et pion. Note : au maximum, il peut y avoir *NB_MAX_DEPL* déplacements possibles pour une pièce donnée. On vérifiera que la case de départ n'est pas vide, et on fera attention aux couleurs des pièces. Si la case de départ est vide, on renverra 0. Si une case parcourue dans la grille est de couleur différente de la case de départ, on peut s'y rendre à condition de respecter le mouvement autorisé par la pièce concernée et de ne pas passer par des cases non vides.

En bonus : ajouter la prise en passant, le roque, la promotion d'un pion. Aussi en bonus, il sera apprécié de tenter au maximum d'éviter la répétition de code dans cette fonction, quitte à introduire de nouvelles fonctions.

```
int listeDeplacementsValides(const char grille[N][N], const int depart[2], int  
deplacements[NB_MAX_DEPL][2]);
```

V - Listes des fonctionnalités en bonus

Vous pouvez réaliser certaines des fonctionnalités listées ci-dessous, ou d'autres à votre convenance afin de valoriser votre travail.

1. Support complet des règles du jeu d'échec :
 - Gérer le roque.
 - La prise en passant.
 - La promotion d'un pion en dame, tour, fou ou cavalier.
 - La règle de la triple répétition et la règle des 50 coups sans capture.
2. Options de jeu et d'affichage :
 - Choix du joueur blanc / noir.
 - Option de tourner l'échiquier lorsque le joueur noir joue.
 - Option d'afficher un roi qui vient d'être mis en échec en rouge.
 - Option de jouer une partie en mode Fischer 960, où la position des pièces au début d'une partie est aléatoire (avec des contraintes).
 - Supporter les parties chronométrées : un joueur perd la partie s'il a épuisé son crédit de temps.
3. Jeu contre une IA :
 - Ajout de l'option de jouer contre une IA qui joue des coups valides, d'abord aléatoirement et éventuellement qui essaie de trouver de bons coups.